Q9. Concept Map – Indexing in DBMS

Q9

INDEXING IN DBMS

Indexing is a technique used to improve the speed of data retrieval operations on a database table.
An index is an auxiliary data structure that allows faster access to rows.

WHAT IS AN INDEX?

An index is an ordered structure that stores the values of one or more columns along with pointers to the actual data in the table.

ADVANTAGES

- Speeds up data retrieval
- Reduces disk I/O
- Improves efficiency of search and sort operations
- Can enforce uniqueness (in case of Unique Index)

TYPES OF INDEXES

1. PRIMARY INDEX

Created automatically on the PRIMARY KEY column.

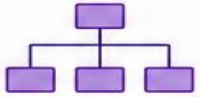
Example: On column StudentID (Primary Key)

2. UNIQUE INDEX

Ensures all values in the indexed column are unique.

Example: On column Email (Unique)

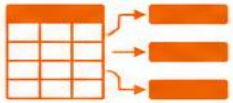
3. CLUSTERED INDEX



Determines the physical order of data in the table.

Example: On column RollNo

4. NON-CLUSTERED INDEX



Does not change the physical order of data. Contains a pointer to the actual data.

Example: On column Name

5. COMPOSITE INDEX

Created on two or more columns.

Example: On columns (FirstName, LastName)

DISADVANTAGES

- Consumes extra storage space
- Slows down data modification operations (INSERT, UPDATE, DELETE)
- Requires additional maintenance

USES OF INDEXES

- To speed up SELECT queries
- To improve performance of JOIN operations
- To enforce PRIMARY KEY and UNIQUE constraints
- To optimize ORDER BY, GROUP BY and WHERE clause operations

COMPARISON OF INDEX TYPES

Index Type	Unique Values	Physical Order of Data	Number of Indexes	Use Case	Performance Impact
Primary Index	Unique	Depends	One per table	Primary Key	High
Unique Index	Unique	No	Multiple	Unique values	High
Clustered Index	Can be Duplicates	Yes (Data sorted)	One per table	Range queries, ORDER BY	Very High
Non-Clustered Index	Can be Duplicates	No	Multiple	General search	High
Composite Index	Depends	No	Multiple	Multi-column search	High

KEY TAKEAWAYS

- Indexes provide faster access to data by reducing the number of disk I/O operations.
- Choose indexes carefully to balance read performance and write overhead.
- Too many indexes can slow down INSERT, UPDATE and DELETE operations.
- Use indexes on columns that are frequently used in WHERE, JOIN, ORDER BY and GROUP BY clauses.